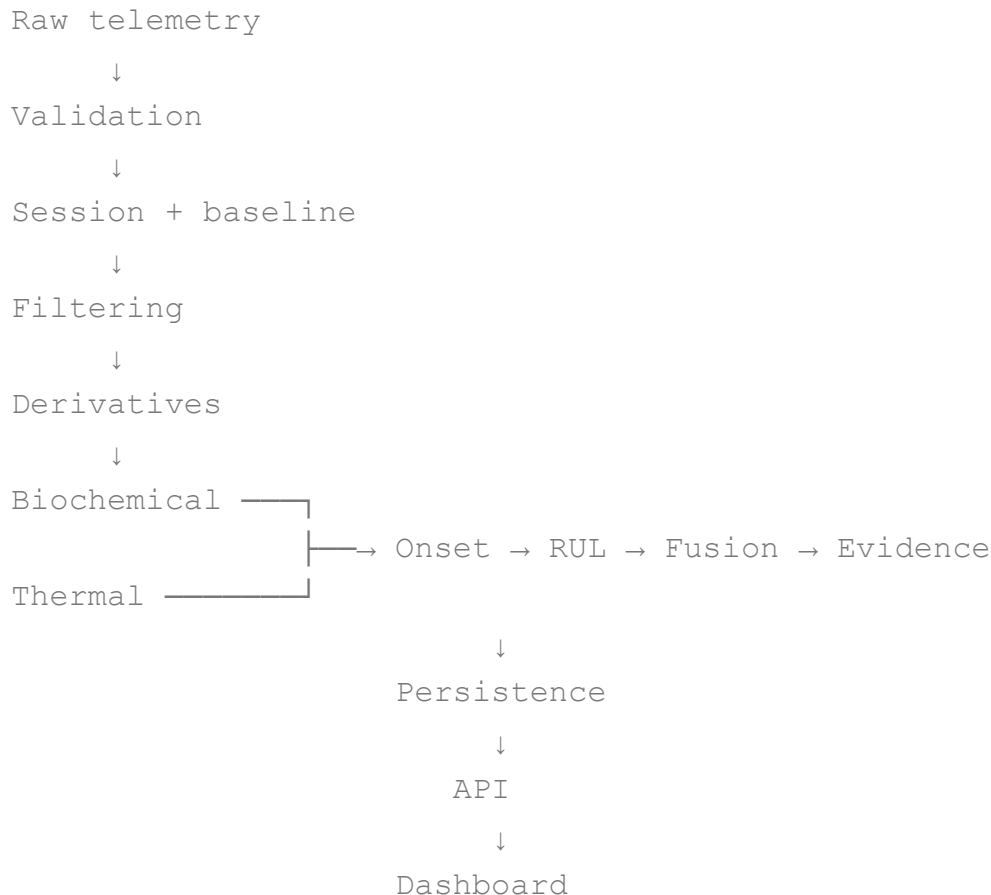


Axios – Coding Plan

0. Our development rule

We build **one vertical pipeline locally first**.



AWS comes **after the local pipeline is proven**.

We do not start by configuring 10 AWS services.

Phase A – Project foundation

Step 1 – Create the repository structure

We'll start with:

```
Axios/
├── src/
│   ├── contracts/
│   ├── ingestion/
│   ├── processing/
│   ├── biochemical/
│   ├── thermal/
│   ├── detection/
│   ├── rul/
│   ├── fusion/
│   ├── evidence/
│   ├── persistence/
│   └── api/
├── tests/
│   ├── unit/
│   ├── contract/
│   ├── replay/
│   ├── integration/
│   └── failure/
├── data/
│   ├── fixtures/
│   ├── raw/
│   └── expected/
├── config/
│   └── food_types/
├── scripts/
├── dashboard/
├── requirements.txt
├── .env.example
├── .gitignore
├── Dockerfile
└── docker-compose.yml
```

But we won't fill all of this immediately.

We'll create the directories as we reach them.

Phase B – Lock the data contracts

This is the **first actual coding task**.

We define the objects that every subsequent phase consumes.

1. RawTelemetry

Something like:

```
RawTelemetry(  
    device_id,  
    session_id,  
    received_at,  
    device_reported_at,  
    food_type,  
    temperature_c,  
    humidity_pct,  
    pressure_hpa,  
    voc_raw,  
    nox_raw,  
    voc_index,  
    nox_index,  
    wifi_rssi,  
    sequence_number,  
)
```

The roadmap explicitly defines this raw structure and says the raw telemetry is the source of truth.

2. ProcessedReading

Contains:

```
raw  
filtered  
baseline  
velocity  
acceleration  
relative deviation  
reading_quality
```

3. Session

Contains:

```
session_id
device_id
food_type
started_at
baseline_established_at
status
baseline statistics
```

4. EvidenceEvent

Contains:

```
event
session_id
timestamp
severity
evidence
```

5. State

Eventually:

```
state
spoilage_risk
onset
RUL
thermal risk
biochemical risk
dominant risk
data quality
```

We **do not implement scoring yet**.

We're just locking interfaces.

Phase C – Build the replay system

This is extremely important.

Before connecting anything to AWS, we need:

```
JSONL telemetry
    ↓
replay_session.py
    ↓
Axios pipeline
```

Example:

```
python scripts/replay_session.py \
    --input data/fixtures/session_001.jsonl
```

This lets us repeatedly run:

```
same input
    ↓
same pipeline
    ↓
compare output
```

This is what allows us to tune filtering, onset and RUL without constantly needing the physical sensor.

The roadmap specifically makes replayability a core architectural requirement.

Phase D – Phase 10: ingestion

Now we actually process telemetry.

P10.1 Validation

Implement:

```
required fields
type validation
temperature range
humidity range
VOC/NOx validity
pressure range
```

Invalid readings become:

```
rejected
+
reason
```

Never silently throw them away.

P10.2 Ordering

Implement:

```
sequence tracking
duplicate detection
out-of-order detection
gap detection
```

Example:

```
100
101
102
104
```

→

```
GAP_DETECTED
missing 103
```

P10.3 Raw storage

Initially:

```
data/raw/  
  device_id/  
    session_id.jsonl
```

Nothing more complicated.

Once this works locally, we can map it to S3.

Phase E – Phase 11: signal processing

This is probably the most important technical phase before scoring.

We'll implement in this order:

P11.1 Session lifecycle

```
SESSION START  
  ↓  
collect baseline  
  ↓  
baseline established  
  ↓  
monitor
```

The roadmap proposes a fixed baseline window such as the first 60 readings or first 10 minutes, whichever is longer.

P11.2 Baseline

Calculate:

```
VOC mean/std  
NOx mean/std  
temperature mean/std
```

per session.

P11.3 Filtering

Pipeline:

```
raw
  ↓
moving median
  ↓
EMA
  ↓
filtered
```

Raw values remain untouched.

P11.4 Derivatives

Then:

```
filtered
  ↓
velocity
  ↓
acceleration
```

using actual Δt , not assuming a fixed sampling interval.

P11.5 Relative deviation

$$D(t) = (x(t) - x_0) / (|x_0| + \varepsilon)$$

Then generate the complete:

```
ProcessedReading
```

Phase F – Phase 12: biochemical engine

Only after Phase 11 is stable.

We'll calculate:

VOC deviation

NOx deviation

VOC slope

NOx slope

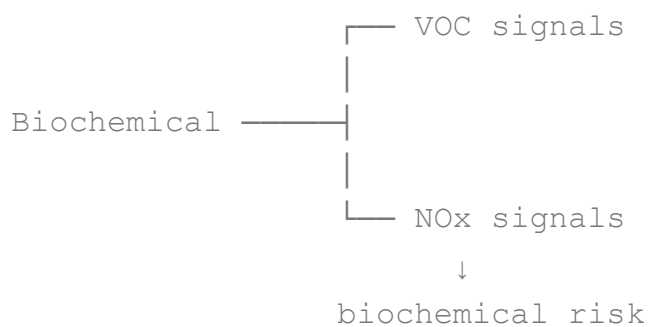
VOC acceleration

NOx acceleration

volatility

persistence

Then:



The roadmap explicitly keeps this deterministic rather than ML-based so every score remains inspectable.

We'll initially use configurable weights.

We won't waste time trying to discover "perfect" weights before we have labeled data.

Phase G – Phase 13: thermal engine

Independent from biochemical.

Implement:

temperature

↓

Kelvin conversion

↓

```
Arrhenius relative rate
    ↓
trapezoidal cumulative exposure
    ↓
thermal risk
```

Food-specific parameters go into:

```
config/food_types/paneer.yaml
```

not Python code.

Phase H – Phase 14: onset

Now we have the signals necessary for onset detection.

Implement the state machine:

```
STABLE
    ↓
WATCH
    ↓
ONSET CANDIDATE
    ↓
SPOILAGE ONSET
    ↓
DEGRADING
```

With:

```
persistence
hysteresis
SNR
engineered confidence
```

The important thing here is that we're **not** doing:

```
if risk > 0.7:
```

```
spoiled = True
```

We're building a state machine that requires sustained evidence.

Phase I – Phase 15: RUL

Only after onset/trajectory signals exist.

We'll start simple:

```
recent degradation trajectory
      ↓
regression
      ↓
slope
      ↓
estimated remaining time
      ↓
uncertainty bounds
```

Then incorporate thermal acceleration.

Output:

```
RUL = 38 h
lower = 27 h
upper = 56 h
```

Not:

```
RUL = 38.42173 h
```

The roadmap explicitly requires a range rather than a bare point estimate.

Phase J – Phase 16: fusion

Very small module.

```
biochemical risk
      +
thermal risk
      ↓
overall risk
      ↓
dominant risk
```

States:

```
LOW_RISK
THERMAL_DOMINANT
BIOCHEMICAL_DOMINANT
DUAL_RISK
INSUFFICIENT_DATA
```

Phase K – Phase 17: evidence

Now we turn the pipeline into something auditable.

Whenever something meaningful happens:

```
BASELINE_ESTABLISHED
TEMPERATURE_EXCURSION_START
TEMPERATURE_EXCURSION_END
BIOCHEMICAL_ACCELERATION
SPOILAGE_ONSET_CANDIDATE
SPOILAGE_ONSET_CONFIRMED
THERMAL_RISK_HIGH
BIOCHEMICAL_RISK_HIGH
RUL_THRESHOLD_CROSSED
DATA_GAP_DETECTED
```

we create an immutable event.

Example:

```
{
  "event": "BIOCHEMICAL_ACCELERATION",
```

```
{
  "session_id": "...",
  "timestamp": "...",
  "severity": 0.82,
  "evidence": {
    "voc_slope": 0.021,
    "nox_slope": -0.045,
    "persistence_seconds": 240,
    "temperature_c": 31.4
  }
}
```

This is a major part of Axios's evidence/audit design.

Phase L – Local persistence + API

Only after the actual inference pipeline works.

Initially:

PostgreSQL

Tables:

```
raw_telemetry
processed_readings
sessions
state_snapshots
events
```

Then FastAPI:

```
GET /devices
GET /devices/{id}/status
GET /devices/{id}/telemetry
GET /devices/{id}/risk
GET /devices/{id}/events
GET /devices/{id}/timeline
```

```
POST /sessions
POST /sessions/{id}/close
```

The API **reads persisted state**.

It does not execute the inference pipeline on every request.

Phase M – Dashboard

Only now.

We need only the essential screens:

Shipment overview

```
Device
Food
Current state
RUL range
Biochemical risk
Thermal risk
Last reading
```

Timeline

```
temperature
VOC
NOx
risk
onset
events
```

Data quality

```
LIVE
STALE
```

INSUFFICIENT DATA
GAP DETECTED

No elaborate frontend architecture until the API is stable.

Phase N – Docker

Once the local pipeline works:

```
Dockerfile
docker-compose.yml
```

Run:

```
processor
postgres
api
```

locally.

This gives us a deployment artifact that can eventually become:

```
Docker
↓
ECR
↓
Fargate
```

Phase 0 – AWS integration

Only now.

We'll integrate incrementally.

AWS step 1

ESP32



IoT Core



Lambda validator



DLQ

AWS step 2

IoT



Firehose



S3

AWS step 3

IoT



Kinesis



Fargate

AWS step 4

Move persistence:

local PostgreSQL



Aurora

and add the appropriate AWS stores where they provide value:

DynamoDB

Timestream

EventBridge

The supplied AWS architecture maps these services to hot state, time-series telemetry, relational state and event fan-out respectively.

Phase P – Cloud E2E test

Then we perform one complete controlled run:

```
ESP32
  ↓
IoT Core
  ↓
validation
  ↓
Kinesis
  ↓
Fargate
  ↓
processing
  ↓
AWS persistence
  ↓
API
  ↓
dashboard
```

And compare the result with the **same session replayed locally**.

That local-vs-cloud comparison is one of the most valuable tests we'll have.

What we are NOT doing

To keep the project lean, we are explicitly **not** starting with:

- ✗ Bedrock
- ✗ Kubernetes
- ✗ microservices everywhere
- ✗ complex CI/CD
- ✗ multi-region

- ✗ autoscaling engineering
- ✗ Step Functions
- ✗ elaborate authentication
- ✗ Slack integrations
- ✗ fancy dashboard
- ✗ ML-based spoilage model
- ✗ premature optimization

Those can be added if the core system is finished and time remains.

The actual coding order

So our immediate execution sequence is:

1. Repository setup
- ↓
2. Python environment
- ↓
3. Data contracts
- ↓
4. Telemetry fixtures
- ↓
5. Replay runner
- ↓
6. Phase 10 validator
- ↓
7. Phase 10 ordering/gaps
- ↓
8. Phase 10 raw JSONL storage
- ↓
9. Phase 11 session/baseline
- ↓
10. Phase 11 filtering
- ↓
11. Phase 11 derivatives
- ↓
12. Phase 11 replay tests
- ↓
13. Phase 12 biochemical
- ↓
14. Phase 13 thermal

↓
15. Phase 14 onset
↓
16. Phase 15 RUL
↓
17. Phase 16 fusion
↓
18. Phase 17 evidence
↓
19. PostgreSQL
↓
20. FastAPI
↓
21. React dashboard
↓
22. Docker
↓
23. AWS integration
↓
24. Cloud E2E
↓
25. Phase 20 validation

The first milestone

We should **not touch AWS yet**.

Our first concrete milestone is:

Given a JSONL file containing real/captured SGP41 telemetry, Axios can validate it, detect bad readings/gaps, store the raw data, establish a session baseline, and produce deterministic processed readings.

Once that works, we have the foundation for everything else.

So the first coding task is the repository + data-contract layer. We should build that before writing the Phase 10 validator.